TS. Please:
- Change all words underlined in BLUE to have initial cap
- Lowercase all words underlined in GREEN
- Number Examples, Theorems, etc. decimally by chapter in a single sequence, e.g. Example 3.1, Definition 3.2, Theorem 3.3, Remark 3.4 etc.
- Examples/Definitions/Claims/Remarks should be set in roman. Propositions/Theorems/Corollaries/Lemmas should remain italicised thoughout. Please amend.

~~Chapter~~ 2

Preface to the Planning Chapters

In the following chapters, we discuss the *planning* problem where a model is known. Before diving in, however, we shall spend some time on defining the various approaches to modeling a sequential decision problem~~,~~ and motivate our choice to focus on some of them. In the next chapters, we will rigorously cover selected approaches and their implications. This chapter is quite different from the rest of the book, as it discusses epistemological and philosophical issues more than anything else.

We are interested in sequential decision problems in which a sequence of decisions need to be taken to achieve a goal or optimize some performance measure. Some examples include:

Example 2.1 (Board games)**.** *An agent playing a board game such as Tic-Tac-Toe, chess~~,~~ or backgammon. Board games are typically played against an opponent~~,~~ and may involve external randomness such as the dice in backgammon. The goal is to play a sequence of moves that lead to winning the game.*

Example 2.2 (Robot Control)**.** *A robot needs to be controlled to perform some task, for example, picking up an object and placing it in a bin, or folding up a piece of cloth. The robot is controlled by applying voltages to its motors, and the goal is to find a sequence of controls that perform the desired task within some time limits.*

Example 2.3 (Inventory Control)**.** *Inventory control is a classical and practical application of sequential decision-making under uncertainty. In its simplest form, a decision maker must determine how much inventory to order at each time period to meet uncertain future demand while balancing ordering costs, holding costs~~,~~ and stockout penalties. The uncertainty in the demand requires a good policy to adapt to the stochastic nature of customer behavior while accounting for both immediate costs and future implications of current decisions. The (s, S) policy, also known as*

a reorder point-order-up-to policy ([128]), is an elegantly simple yet often optimal approach to inventory control. Under this policy, whenever the inventory level drops to or below a reorder point s, an order is placed to bring the inventory position up to a target level S. While finding the optimal values for s and S is non-trivial, this policy structure has been proven optimal for many important inventory problems under reasonable assumptions. The (s, S) framework provides an excellent example of how constraining the policy space, in this case to just two parameters, can make learning more efficient while still achieving strong performance.

When we are given a sequential decision problem, we have to model it from a mathematical perspective. In this book, and in much of the literature, the focus is mostly on the celebrated Markov Decision Process (MDP) model. It should be clear that this is merely a model, i.e., one should not view it as a precise reflection of reality. To quote Box: "*all models are wrong, but some are useful*". Our goal is to have useful models, and as such the MDP model is a perfect example. The MDP model has the following components, which we discuss here and provide formally in later chapters. We will use the agent-centric view, assuming an agent interacts with an environment. This agent is sometimes called a "decision maker", especially in the operations research community.

1. States: A state is the atomic entity that represents all the information needed to predict future rewards of the system. The agent in an MDP can fully observe the state.

2. Actions: An action is what can be affected by the decision maker.

3. Rewards: The rewards represent some numerical measurement that the decision maker wishes to maximize. The reward is assumed to be a function of the current state and the action.

4. Dynamics: The state changes (or transitions) according to the dynamics. This evolution depends only on the current state and the action chosen but not on future or past states or actions.

In planning, it is assumed that all the components are known. The objective of the decision maker is to find a policy, i.e., a mapping from histories of state observations to actions, that maximizes some objective function of the reward. We will adopt the following standard assumptions concerning the planning model:

1. Time is discrete and regular: decisions are made in some predefined decision epochs. For example, every second/month/year. While continuous time is

especially common in robotic applications, we will adhere for simplicity to discrete regular times. In principle, this is not a particularly limiting assumption, as most digital systems inherently discretize the time measurement. However, it may be unnecessary to apply a different control at *every* time step; The semi-MDP model is a common framework to use when the decision epochs are irregular [121], and there is an extensive literature on optimal control in continuous time [80], which we will not consider here.

2. Action space is finite. We will mostly assume that the available actions a decision maker can choose from belong to a finite set. While this assumption may appear natural in board games, or any digital system that is discretized, in some domains such as robotics it is more natural to consider a continuous control setting. For continuous actions, the *structure* of the action space is critical for effective decision making – we will discuss some specific examples, such as a linear dynamical system here. More general continuous and hybrid discrete-continuous models are often studied in the control literature [16] and in operations research [120].

3. State space is finite. The set of possible system states is also assumed to be finite and unstructured. The finiteness assumption is mostly a convenience, as any bounded continuous space can be finely discretized to a finite, but very large set. Indeed, in the second part of this book, we shall study learning-based methods that can handle very large state spaces. For problems where the state space has a known and convenient structure, a model that takes this structure into account can be more appropriate. For example, in a linear controlled dynamical system, which we discuss in Section 3.6, the state space is continuous, and its evolution with respect to the control is linear, leading to a closed form optimal solution when the reward has a particular quadratic structure. In the classical STRIPS and PDDL planning models, which we do not cover here, the state space is a list of binary variables (e.g., a system for robot setting a table may be described by [robot gripper closed = False, cup on table = True, plate on table = False,. . .]), and planning algorithms that try to find actions that lead to certain goal states being 'true' can take account of this special structure [126].

4. Rewards are all given in a single currency. We assume that the agent has a single reward stream it tries to optimize. Specifically the agent tries to maximize the long term sum of rewards. In some cases, a user may be interested in other statistics of the reward, such as its variance [103], or to balance multiple

types of rewards [102]; we do not cover these cases here.

5. Decision Horizon or Discounting is known. It is assumed in virtually all the literature that the time horizon for the decision process is a known stopping time. That is, the sequential process terminates as a result of an event that depends on history. For discounted problems, the discount factor is assumed to be a part of the problem definition.

6. Markov state evolution. We shall assume that the environment's reaction to the agent's actions is fixed (it may be stochastic, but with a fixed distribution)~~,~~ and depends only on the current state. This assumption precludes environments that are *adversarial* to the agent~~,~~ or systems with multiple independent agents that learn together with our agent [29, 174].

As should be clear from the points above, the MDP model is agnostic to the structure that certain problems may possess~~,~~ and more specialized models may exploit. The reader may question, therefore, why study such a model for planning. As it turns out, the simplicity and generality of the MDP model is actually a blessing when using it for *learning*, which is the main focus of this book. The reason is that the structure of a specific problem may be implicitly picked up by the learning algorithm, which is designed to identify patterns in data. This strategy has proved to be very valuable in computing decision making policies for problems where structure exists~~,~~ but is hard to define manually, which is often the case in practice. Indeed, many recent RL success stories, such as mastering the game of Go, managing resources in the complex game of StarCraft, and state-of-the-art continuous control of racing drones, have all used the simple MDP model combined with powerful deep learning methods [138, 170, 72].

There are two other strong modelling assumptions in the MDP model: (1) all uncertainty in decision making is limited to the randomness of the (Markov) state transitions, and (2) the objective can only be specified using rewards. Next, we discuss these two design choices in more detail.

2.1 Reasoning Under Uncertainty

A main objective of planning and learning is to facilitate reasoning under uncertainty. Uncertainty can come in many forms, as humans often encounter every day: when playing a board game, we do not know what our opponent will do; when rolling a dice, we do not know what ~~will be~~ the outcome; when folding laundry, it is likely

that an accurate physical model of the cloth is not available; when driving in a city, we may not observe cars hidden behind a corner; etc.

Here we list common types of uncertainty and, afterwards, discuss how they relate to planning with MDPs.

Aleatoric[1] *uncertainty*: Handling inherent randomness that is part of the model. For example, a model can contain an event which occurs with a given probability. For example, in the board game backgammon, the probability of a move is given by throwing two dices.

Epistemic[2] *uncertainty*: Dealing with a lack of knowledge about the model parameters. Sometimes we do not know what the exact model parameters are because more interaction is needed to learn them (this is addressed in the learning part of this book). Sometimes we have a nominal model and the true parameters are only revealed at runtime (this is addressed within the robust MDP framework; see [115]). Sometimes our model is too coarse or simply incorrect – this is known as *model misspecification*.

Partial observability: Reasoning with incomplete information concerning the true state of the system. There are many problems where we just do not have an accurate measurement that can help us predict the future and instead we get to observe partial information concerning the true state of the system. Some would argue that all real problems have some elements of partial observability in them.

We emphasize that for planning and for learning, a model could combine all types of uncertainty. The choice of which type of uncertainty is an important design choice.

The MDP model that we focus on in the planning chapter only accommodates aleatoric uncertainty, through the stochastic state transitions. While this may appear to be a strong limitation, MDPs have proven useful for dealing with more general forms of uncertainty. For example, in the learning chapters, we will ask how to update an MDP model from interaction with the environment, to potentially reduce epistemic uncertainty. For board games, even though MDPs cannot model an adversary, assuming that the opponent is stochastic helps find a robust policy against various opponent strategies. Moreover, by using the concept of *self play* – an agent that learns to play against itself and continually improve – RL has produced the most advanced AI agents for several games, including Chess and Go. For partially observable systems, a fundamental result shows that taking the full observation history as the 'state', results in an MDP model for the problem (albeit with a huge state space).

[1]The term comes from the Latin word "alea", which means dice or a game of chance.

[2]The term is derived from the Greek word "episteme", meaning knowledge.

2.2 Objective Optimization

A central assumption in planning is that an immediate reward function is given to the decision maker and the objective is to maximize some sort of expected cumulative discounted (or total or average) reward. From the perspective of the planner, this makes the planner's life easy as the objective is defined in a formal manner. Nevertheless, in many applications, much of the problem is to engineer a "right" reward function. This may be done by understanding the specifications of the problem, or from data of desired behavior, a problem known as *Inverse Reinforcement Learning* [114]. In inverse reinforcement learning the decision maker typically observes trajectories that were sampled by an expert and needs to infer an optimal policy often based on estimating the reward function. While the problem is ill-defined, since many reward functions may lead to similar optimal trajectories, the main issue is to obtain good *generalization* and for that robustness is key.

In some cases, the reward stream is very sparse. For example, in board games the reward is often obtained only at the end of the game in the form of a victory or a loss. While this does not pose a conceptual problem, it may lead to practical problems as we will discuss later in the book. A conceptual solution here is to use "proxy rewards", which are intermediate reward indicating progress towards a desired goal.

A limitation of the MDP planning model is the underlying assumption that preferences can be succinctly represented through reward functions. Although in principle any preference among trajectories can be represented using a reward function, by extending the state space to include all history, this may be cumbersome and may require a much larger state space. Specifically, the discount factor, which is often assumed a part of the problem specifications, represents a preference between short-term and long-term objectives. Such preferences are often arbitrary.

We finally comment that the assumption that there exists a scalar reward we optimize (through a long term objective) does not hold in many problems. Often, we have several, potentially contradicting, objectives. For example, we may want to minimize power consumption while maximizing throughput in communication networks. In general, part of the reward function engineering pertains to balancing different objectives, even if they are not measured in the same way ("adding apples and oranges"). A different approach is to embrace the multi-objective nature of the decision problems through constrained Markov decision processes [3], or using other approaches [e.g., 102]. Moreover, even when all aspects in expectation of a problem can be amortized with a single reward function the decision maker may need to consider risk issues, such as resilience to rare events. We emphasize that the reward function is a design choice made by the decision maker.

Nevertheless, MDPs with their scalar reward function have proven useful in many practical domains, as the availability of strong algorithms for solving MDPs effectively allows the system engineer to tweak the reward function manually to fit some hard-to-quantify desired behavior.

2.3 Importance of Small (Finite) Models

The next few chapters, and indeed much of the literature, explicitly assume that the models are finite (in terms of actions and states) and even practically small. While this is certainly justified from a pedagogical perspective, there are additional reasons that make small models relevant.

Small models are more interpretable than large ones: it is often the case that a different state captures particular meanings leading to more explainable policies. For example, in inventory control problems, the dynamic programming techniques that we will study can show that for certain simplified problem instances, an *optimal* strategy has the structure of a threshold policy – if the inventory is below some certain threshold then replenish, otherwise do not. Such observations about the structure of optimal policies often inform the design of policies for more complex scenarios.

The language and some fundamental concepts we shall develop for small models, such as the value function, value iteration and policy iteration algorithms, and convergence of stochastic approximation, will also carry over to the learning chapters, which deal with large state spaces and approximations.